DevOps Intern Final Assessment

Name: Yajat Prabhakar
Date: August 18, 2026

Project Overview

This repository demonstrates a small end-to-end DevOps pipeline using open-source tools.

Each stage produces an artifact or result that is consumed by the next stage:

```
Git Repository
      ↓
Shell Script
      ↓
Docker Image
      ↓
GitHub Actions CI
      ↓
Nomad Job
      ↓
Container Logs
      ↓
Grafana Loki
```

The project covers:

- Git & GitHub
- Linux and Bash scripting
- Docker
- CI/CD with GitHub Actions
- Container orchestration with HashiCorp Nomad
- Centralized log collection with Grafana Loki and Promtail

---

1. Git & GitHub Setup

The project is maintained in a public GitHub repository.

The repository contains the source code, shell scripts, Docker configuration, CI workflow, Nomad job specification, and monitoring configuration.

Application

`hello.py` is a simple Python application that prints:

```
Hello, DevOps!
```

Run it locally:

```
python3 hello.py
```

Expected output:

```
Hello, DevOps!
```

---

2. Linux & Bash Scripting

The project includes a Bash script at:

```
scripts/sysinfo.sh
```

The script displays:

- Current user
- Current date and time
- Disk usage

Make the script executable:

```
chmod +x scripts/sysinfo.sh
```

Run it:

```
./scripts/sysinfo.sh
```

Example output:

```
yajat
Mon Aug 18 20:15:02 UTC 2026
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   12G   36G  25% /
```

The exact output will vary depending on the system running the script.

---

3. Docker Containerization

The application is containerized using Docker.

The `Dockerfile` uses the lightweight Python 3.12 Slim image as its base image.

Build the image

```
docker build -t hello-devops:v1 .
```

Run the container

```
docker run --rm hello-devops:v1
```

Expected output:

```
Hello, DevOps!
```

Why `v1` instead of `latest`?

The image is deliberately tagged as:

```
hello-devops:v1
```

rather than:

```
hello-devops:latest
```

This is important for the local Nomad deployment used in this assessment.

Nomad's Docker driver treats the `latest` tag differently and may attempt to pull it from a registry even when a local image exists. Using an explicit version tag such as `v1`, together with:

```
force_pull = false
```

allows Nomad to use the locally available image.

This also follows a better DevOps practice of using explicit image versions rather than relying on the mutable `latest` tag.

Docker verification

```
docker images
```

The resulting image should contain:

```
hello-devops
v1
```

---

4. CI/CD with GitHub Actions

The repository contains the following GitHub Actions workflow:

```
.github/workflows/ci.yml
```

The workflow runs automatically on:

- Pushes to `main`
- Pull requests targeting `main`

CI Pipeline

The workflow performs the following steps:

1. Checks out the repository.
2. Sets up Python 3.12.
3. Executes `hello.py`.
4. Executes `scripts/sysinfo.sh`.

Conceptually:

```
Git Push / Pull Request
          ↓
    GitHub Actions
          ↓
    Checkout Code
```

```
        ↓
  Setup Python 3.12
        ↓
     Run hello.py
        ↓
  Run sysinfo.sh
        ↓
      CI Result
```

After pushing the repository, the workflow status can be verified from the **Actions** tab on GitHub.

---

5. Job Deployment with HashiCorp Nomad

The Docker image created in Step 3 is deployed as a Nomad job.

The job specification is located at:

```
nomad/hello.nomad
```

Environment

This project was developed on Windows using Docker Desktop.

Initially, the native Windows Nomad agent could not communicate correctly with Docker Desktop when Docker Desktop was operating in Linux-container mode. The Docker driver reported an unhealthy state because the native Windows environment expected a Windows-container Docker endpoint.

The issue was resolved by running the Nomad agent inside **WSL2 with Ubuntu**, while using Docker Desktop's WSL integration.

This allows Nomad to communicate with Docker using the standard Linux Docker socket:

```
unix:///var/run/docker.sock
```

No additional Docker driver plugin configuration was required.

Why the Nomad job uses `batch`

The assessment brief suggests a:

```
type = "service"
```

job.

However, this application is a one-shot workload:

```
hello.py
    ↓
prints "Hello, DevOps!"
    ↓
process exits
```

A Nomad `service` job is intended for long-running workloads. Therefore, Nomad attempts to restart the task when the process exits.

Although the Python process exits successfully with:

```
Exit Code: 0
```

a service workload is expected to remain running.

For this reason, the project uses:

```
type = "batch"
```

A `batch` job is appropriate for a finite workload because successful completion is represented as:

```
Client Status = complete
```

This makes the job type consistent with the actual behavior of the application.

Final Nomad Job

```
job "hello" {
  datacenters = ["dc1"]
  type        = "batch"

  group "hello-group" {
    count = 1

    task "hello-task" {
      driver = "docker"

      config {
```

```
        image      = "hello-devops:v1"
        force_pull = false
      }

      resources {
        cpu    = 100
        memory = 128
      }

      restart {
        attempts = 0
        mode     = "fail"
      }
    }
  }
}
```

Prerequisites

Before running the Nomad job:

1. Docker Desktop must be running.
2. WSL2 must be installed and configured.
3. Docker Desktop WSL integration must be enabled for Ubuntu.
4. Nomad must be available inside WSL2.
5. The Docker image must already exist locally.

Build the image:

```
docker build -t hello-devops:v1 .
```

Start a development Nomad agent:

```
nomad agent -dev
```

Run the Nomad Job

From the project directory:

```
nomad job run nomad/hello.nomad
```

Check the job:

```
nomad job status hello
```

Retrieve the allocation ID:

```
nomad job status hello
```

Then inspect the allocation:

```
nomad alloc status <alloc-id>
```

View the application logs:

```
nomad alloc logs <alloc-id>
```

Expected application output:

```
Hello, DevOps!
```

Successful Allocation

The actual deployment completed successfully with:

```
Client Status       = complete
Client Description  = All tasks have completed
```

The task terminated with:

```
Exit Code: 0
```

This confirms that:

- Nomad successfully scheduled the job.
- Nomad successfully accessed the Docker driver.
- The local Docker image was used.
- The container started successfully.
- The Python application completed successfully.

- The allocation was marked `complete`.

6. Monitoring with Grafana Loki

The final stage of the pipeline collects container logs using:

- Grafana Loki
- Promtail
- Grafana

The architecture is:

```
Docker Container
      ↓
Docker Logs
      ↓
   Promtail
      ↓
    Loki
      ↓
   Grafana
      ↓
   Explore
```

Start Loki

Run Loki locally:

```
docker run -d \
  --name=loki \
  -p 3100:3100 \
  grafana/loki:2.9.0 \
  -config.file=/etc/loki/local-config.yaml
```

Verify that Loki is running:

```
docker ps
```

Loki should be available at:

```
http://localhost:3100
```

Start Promtail

Promtail is configured using:

```
monitoring/promtail-config.yaml
```

Run Promtail:

```
docker run -d \
  --name=promtail \
  -v /var/lib/docker/containers:/var/lib/docker/containers:ro \
  -v "$(pwd)/monitoring/promtail-config.yaml:/etc/promtail/config.yml" \
  grafana/promtail:2.9.0 \
  -config.file=/etc/promtail/config.yml
```

Promtail reads Docker container logs and forwards them to Loki.

Query Logs in Grafana

Add Loki as a Grafana data source using:

```
http://localhost:3100
```

Then open:

```
Grafana → Explore
```

A container log query can be performed using:

```
{container_name="hello"}
```

The expected result includes the application's output:

```
Hello, DevOps!
```

This verifies that the application logs successfully travelled through the monitoring pipeline:

```
Application
    ↓
Docker
    ↓
Promtail
    ↓
Loki
    ↓
Grafana
```

---

7. Evidence / Screenshots

The following screenshots can be added to the `docs/` directory to provide visual evidence of each stage.

Docker

```
docs/docker-run.png
```

Should show:

```
docker build -t hello-devops:v1 .
docker run --rm hello-devops:v1
```

and:

```
Hello, DevOps!
```

Nomad

```
docs/nomad-run.png
```

Should show:

- `nomad job status hello`
- Successful allocation
- Exit code `0`
- `nomad alloc logs`
- `Hello, DevOps!`

Grafana Loki

```
docs/grafana-loki.png
```

Should show the Grafana Explore interface with a Loki query such as:

```
{container_name="hello"}
```

and the resulting application log.

---

8. Extra Credit

The optional extra-credit components were not implemented in this submission.

Potential extensions include:

MLflow

```
mlflow/
```

A dummy MLflow experiment could be added to demonstrate experiment tracking.

Virtual Machine Deployment

```
vm/
```

A VirtualBox VM could be provisioned and used to run the Docker/Nomad workload inside a virtualized Linux environment.

---

9. Repository Structure

```
.
├── README.md
├── hello.py
├── Dockerfile
│
├── scripts/
│   └── sysinfo.sh
```

```
│
├── .github/
│   └── workflows/
│       └── ci.yml
│
├── nomad/
│   └── hello.nomad
│
├── monitoring/
│   ├── loki_setup.txt
│   └── promtail-config.yaml
│
└── docs/
    ├── docker-run.png
    ├── nomad-run.png
    └── grafana-loki.png
```

10. End-to-End Verification

The complete pipeline can be verified in the following order.

Step 1 — Run the application

```
python3 hello.py
```

Expected:

```
Hello, DevOps!
```

Step 2 — Run the system information script

```
./scripts/sysinfo.sh
```

Step 3 — Build the Docker image

```
docker build -t hello-devops:v1 .
```

Step 4 — Run the container

```
docker run --rm hello-devops:v1
```

Expected:

```
Hello, DevOps!
```

Step 5 — Verify CI

Push the repository to GitHub and verify the workflow under:

```
GitHub → Actions
```

Step 6 — Deploy with Nomad

```
nomad job run nomad/hello.nomad
```

Step 7 — Verify the allocation

```
nomad job status hello
```

The job should reach:

```
complete
```

Step 8 — Retrieve Nomad logs

```
nomad alloc logs <alloc-id>
```

Expected:

```
Hello, DevOps!
```

Step 9 — Verify Loki

Open Grafana Explore and query:

```
{container_name="hello"}
```

The application log should be visible.

Conclusion

This project demonstrates an end-to-end DevOps workflow using open-source tooling.

The final pipeline connects source code, automation, containerization, job scheduling, and monitoring:

```
GitHub
  ↓
GitHub Actions
  ↓
Python Application
  ↓
Docker Image
  ↓
Nomad Batch Job
  ↓
Docker Container
  ↓
Promtail
  ↓
Grafana Loki
  ↓
Grafana
```

The implementation also addresses practical deployment issues encountered during development, including Docker/Nomad integration under Windows, WSL2-based execution, appropriate Nomad job types for finite workloads, and explicit Docker image versioning.

Assessment status: Core DevOps pipeline implemented successfully.